ROBOTIC ARM CONTROL APP

10-page master product, engineering and execution plan



PRODUCT NORTH STAR

A small, polished Android controller for the actual 6-DOF arm. The app must feel like a real robotics controller, not a generic college dashboard. The non-negotiable functions are: (1) game-like manual control, (2) clear and trustworthy joint angles, (3) reliable HC-05 Bluetooth Classic communication, and (4) safe stopping/limits.

LOCKED DESIGN RULES

• Visual: use the uploaded reference only for the soft light-blue/cyan/white visual language, spacing, cards, typography and product feel; use our real 3D arm as the hero visual.

• Never: purple gradients, excessive glassmorphism, pill-button UI, fake metrics/reviews, vague marketing copy, emoji as UI icons, cursor animation, AI-slop photos/copy, fake customer counters or generator branding.

• Motion: short purposeful transitions only. Launch animation is a 3–5 second pick-and-place demonstration; no long cinematic sequence or excessive scroll effects.

• Control first: manual control is the default and highest-priority workflow. Angle/Pose mode is the second core workflow.

• Release gate: do not call the product final until the required domain, favicon, privacy policy, terms page and branding cleanup are complete.

SOURCE-OF-TRUTH: current plan already locks Flutter Android + HC-05 Classic, manual game-like control, six-angle target control, safety, and Firebase only as a supporting layer. filecite turn0file0 L684-L735

01 • SYSTEM SCOPE & HARDWARE CONTRACT

The physical system is already built. The app should adapt to the hardware rather than forcing a new hardware architecture.

Layer	Current system	Responsibility
Mobile	Flutter / Dart Android APK	UI, Bluetooth, commands, pose entry, presets, visualization
Wireless	HC-05 Bluetooth Classic / SPP	Phone ↔ Arduino serial transport
Controller	Arduino UNO R3	Command parser, safety, real-time motion, servo state
PWM	PCA9685 16-channel servo driver	Generates servo PWM
Actuators	MG995, MG996R, SG90	Physical joints / gripper
Cloud	Firebase / Firestore (optional support)	Saved poses, sequences, settings, optional history

LOCKED JOINT MAP

Channel	Joint	Servo	App name
CH0	Base	MG995	Base
CH1	Shoulder	MG996R	Shoulder
CH2	Upper arm / elbow	MG996R	Elbow
CH3	Lower arm	SG90	Lower Arm
CH4	Wrist	SG90	Wrist
CH5	Gripper	SG90	Gripper

POWER / ELECTRICAL GATE

• Current architecture uses an external 5 V high-current supply for the PCA9685 servo rail; never power the servo rail from Arduino USB/logic power.

• Common ground: Arduino GND, PCA9685 GND and HC-05 GND must share reference.

• Before full simultaneous movement, verify the actual supply voltage under load, connector heating and servo peak/stall current. The nominal 5 V/10 A supply is the current project configuration, not a guarantee of peak-demand margin.

• Use a proper bulk capacitor near the PCA9685 servo supply and secure/strain-relieve the temporary jumper wiring before full-load demonstrations.

ENGINEERING RULE: software cannot compensate for a weak power path. A Bluetooth or UI problem is acceptable during development; uncontrolled servo movement or undervoltage is not.

02 • CONTROL ARCHITECTURE & DATA FLOW

Real-time motion must stay local. Firebase is never inserted between a touch action and the servo.

Path	Flow	Purpose
Manual	Flutter → HC-05 → UNO → PCA9685 → servo	Lowest-latency live control
Pose	Flutter validates target → UNO receives target → UNO interpolates → servos	Coordinated target movement
Status	UNO → HC-05 → Flutter	Connection + commanded joint state
Cloud	Flutter ↔ Firestore	Presets, sequences, settings; never real-time motor control

RESPONSIBILITY BOUNDARIES

Flutter	Arduino	Firebase
UI state	Servo PWM	Saved poses
U	s	e
C	o	m
S	e	q
B	l	u
J	o	i
S	e	t
T	a	r
E	m	e
O	p	t
P	r	e
M	o	t
U	s	e
V	i	s
C	o	n
N	o	t

STATE MODEL

RobotState: connectionState, robotReady, emergencyStopped, motionState, currentCommandedAngles[6], targetAngles[6], speedProfile, jointLimits[6], selectedJoint, lastError. Important: without physical feedback sensors, 'current angle' is the last commanded/calibrated servo position, not a measured shaft angle. The UI must not claim sensor-level accuracy.

UI STATE MACHINE

Disconnected → Pairing → Connecting → Handshake → Ready → Moving → Reached/Idle. Any transport loss or safety fault forces Stopping → Disconnected/Fault. The controller must reject movement commands unless the robot is Ready.

The locked architecture explicitly assigns UI/Bluetooth/targets to Flutter, real-time movement/safety to Arduino, and saved poses/sequences to Firebase. filecite turn0file0 L605-L660

03 • BLUETOOTH, PROTOCOL & SAFETY

Use Bluetooth Classic SPP because the current HC-05 is a Classic serial module. A current Flutter option is flutter_classic_bluetooth, which exposes RFCOMM/SPP discovery, pairing, connection and byte-stream I/O and explicitly supports HC-05/Arduino. It currently requires Flutter 3.44+ according to its package page. □cite□turn0search0□turn0search1□

CONNECTION FLOW

Open app → check Bluetooth → request permissions → show paired/scan devices → select HC-05 → connect → handshake → request status → enable controls. Do not show 'Ready' merely because the Bluetooth socket is connected.

HANDSHAKE

APP → HELLO\nUNO → ROBOT_6DOF_READY v1\nAPP → STATUS?\nUNO → STATE ...\nAPP → READY_ACK\nUNO → OK

COMMAND PROTOCOL V1

Command	Example	Meaning
Manual joint	J+ / J-	Move selected joint in positive/negative direction while held
Select joint	SEL:0..5	Select Base..Gripper for fine mode
Speed	SPD:1/2/3	Slow / Normal / Fast
Target pose	POSE:90,75,110,90,85,25	Set six target angles
Execute	MOVE	Arduino interpolates to target
Stop	STOP	Stop active motion
Emergency	ESTOP	Immediate software motion stop + fault state
Home	HOME	Move to validated home pose
Status	STATUS?	Return state and commanded angles
Ping	PING	Link health test

MANUAL SAFETY CONTRACT

- Joystick/button movement is momentary: press/hold = move; release = stop.
- App applies limits, but Arduino applies the authoritative limits again.
- If Bluetooth disconnects during movement, Arduino must time out and stop motion.
- Every command is validated for range and format. Invalid commands are ignored and reported.
- ESTOP remains visible in the control UI and does not depend on Firebase.
- Target motion uses bounded interpolation, not abrupt jumps.

The earlier plan explicitly requires handshake, joint limits, Arduino-side safety and a permanent STOP ALL path. □filecite□turn2file9□

04 • UX JOURNEY & VISUAL SYSTEM

The uploaded reference defines the visual mood: soft light blue/cyan/white, clean product cards, strong imagery, clear typography and generous spacing. We replace the reference robot with our actual 3D arm.

LAUNCH EXPERIENCE

Frame 1 • logo/arm mark appears → Frame 2 • real 3D arm fades in at Home → Frame 3 • small block appears → Frame 4 • arm performs a short pick-and-place → Frame 5 • arm settles → Frame 6 • dashboard appears. Duration: 3–5 seconds. Provide a reduced-motion/static fallback.

No cinematic camera spins, particle effects, cursor animation or decorative motion. Every animation must communicate state or product identity.

VISUAL TOKENS

Element	Decision
Primary palette	Soft sky blue / cyan accents + white surfaces + deep blue-gray text
Background	Very light blue/white; no purple gradient
Cards	Moderate corner radius, thin border, restrained shadow; never every element in a separate card
Buttons	Rectangular/soft-rounded controls with clear labels; no excessive pill shapes
Typography	One clean sans family; strong numeric hierarchy for angles
Icons	Material/SF-style vector icons; no emoji used as interface icons
Motion	150–350 ms micro-transitions; 3–5 s launch only
3D asset	Our actual arm model/render; optimized for mobile
Feedback	Color + text + icon state; never color alone

NAVIGATION

Home = status + arm + quick actions. Control = primary game-like manual control. Pose = precise six-angle target control. Saved = poses/sequences. Settings = Bluetooth, calibration, limits, legal pages.

The visual rules are locked: the reference is inspiration only; no AI-looking imagery/copy, purple gradients, fake metrics/reviews, vague hero text, emoji UI, cursor animation or excessive scrolling effects. filecite turn1file0 L5-L40

05 • MANUAL GAME CONTROL — CORE FEATURE

This is the app's primary screen. It must be usable while the operator watches the physical arm instead of staring at the phone.

CONTROL MAPPING

Physical control	Joint	Behavior
Left joystick X	Base	Left/right rotation; dead-zone near center
Left joystick Y	Shoulder	Up/down movement; speed proportional to displacement
Right joystick X	Elbow	Forward/back angular movement
Right joystick Y	Lower Arm	Up/down angular movement
Wrist rocker	Wrist	Dedicated left/right or up/down fine control
Gripper	Gripper	Open/close dedicated control; optional hold-to-move
Fine Joint mode	Any selected joint	Single-axis precision control

GAME-LIKE BEHAVIOR

- Use a configurable joystick dead-zone around center to prevent servo chatter.
- Map displacement to speed: small displacement = slow precision movement; full displacement = selected maximum speed.
- Release/return to center = stop command. Do not rely on the user pressing a separate stop after every movement.
- Add a small 'Fine' toggle for slow precision movement when positioning the arm near an object.
- Show the five arm joint angles plus gripper angle continuously in a compact status strip.
- Keep STOP ALL fixed/always reachable and visually distinct from ordinary controls.

CONTROL SCREEN COMPOSITION

TOP: connection + robot ready state\nCENTER: actual 3D arm + compact live angle list\nLOWER: two large joysticks + wrist/gripper controls\nBOTTOM: STOP ALL + Home/Fine/Speed\n

Do not fill the screen with six independent servo cards. The earlier plan deliberately moved toward a two-stick game-controller interaction, with fine-control mode for individual joints. filecite turn2file7

06 • ANGLE / POSE MODE & MOTION

Pose mode is the second non-negotiable feature: the operator enters six target angles, reviews them, then explicitly commands the robot to move.

TARGET EDITOR

Joint	Current*	Target	Limit
Base	92°	120°	calibrated
Shoulder	74°	75°	calibrated
Elbow	108°	110°	calibrated
Lower Arm	90°	90°	calibrated
Wrist	85°	85°	calibrated
Gripper	27°	30°	calibrated

*Current = last commanded/calibrated position unless feedback sensors are later added. Never label open-loop commanded position as measured physical position.

POSE WORKFLOW

1. Edit → 2. Validate limits → 3. Preview arm pose → 4. Confirm MOVE TO TARGET → 5. Arduino interpolates → 6. App receives progress/status → 7. Target reached.

COORDINATED MOTION

Arduino calculates the longest required joint travel and uses it to establish a common motion duration. Each joint interpolates from current commanded angle to target so all joints arrive together. Add acceleration/deceleration or small step smoothing only after basic synchronized motion is stable.

CALIBRATION

- Define mechanical home physically for every joint.
- Store per-joint offset and safe min/max angle.
- Test each joint in isolation at low speed.
- Record the servo command that corresponds to the real mechanical reference mark.
- Never assume all servos have identical 0°/90° geometry after installation.
- Persist calibration locally; optionally back up settings in Firestore.

POSES / PRESETS

Save current pose, rename it, preview it, and Move. Initial presets: Home, Pick, Lift, Drop. Sequence/teach mode is an add-on only after manual and pose control are stable.

The project plan explicitly requires target preview, coordinated movement and Arduino-side interpolation rather than independent abrupt servo jumps. filecite turn2file5

07 • FLUTTER APP ARCHITECTURE & IMPLEMENTATION

Use a small clean architecture: presentation, application/controller state, hardware transport, domain models, persistence. Avoid over-engineering a college project.

PROJECT STRUCTURE

lib/\n core/ theme, constants, safety, utilities\n models/ joint, robot_state, pose, sequence\n bluetooth/ manager, protocol, connection state\n controllers/ robot_controller, motion_controller, safety_controller\n services/ firestore_service, preset_service\n screens/ splash, home, control, pose, saved, settings\n widgets/ joystick, angle_row, robot_view, status, estop\n main.dart\n

RECOMMENDED STACK

Area	Choice	Reason
UI	Flutter + Dart	Single Android codebase, custom interaction/animation
State	Riverpod or Provider	Simple reactive robot state; avoid global mutable UI state
Bluetooth	flutter_classic_bluetooth (current candidate)	Classic RFCOMM/SPP; current package explicitly supports HC-05/Arduino; verify package at in
Cloud	Firebase Auth + Firestore	Only for saved data/supporting features
Animation	Flutter animation / Rive or Lottie	Short controlled launch transition
3D	Optimized pre-rendered/interactive asset	Use actual arm; keep mobile performance stable

IMPLEMENTATION ORDER

Phase 0 — Freeze: joint map, electrical wiring, safe limits, protocol.

Phase 1 — Arduino: command parser, status, stop, limits, single-joint testing, coordinated pose motion.

Phase 2 — Bluetooth: Flutter discovery/pairing/connect, handshake, send/receive, reconnect and timeout.

Phase 3 — Core UI: design system, splash, dashboard, manual control, live angles.

Phase 4 — Pose: six-angle editor, validation, preview, MOVE, calibration.

Phase 5 — Persistence: saved poses; sequences only if time remains.

Phase 6 — Polish: 3D arm, motion, accessibility, error states, performance.

Phase 7 — Release: APK, domain pages, favicon, privacy, terms, final hardware validation.

The existing project architecture already separates Flutter, Bluetooth, controllers, Firebase and screens/widgets rather than putting everything in main.dart. filecite turn2file3

08 • TESTING, ACCEPTANCE & RELEASE GATES

We do not judge the app by how attractive the UI looks until the physical control loop is reliable.

COMMISSIONING TEST ORDER

Test	Pass condition
Power only	Arduino/PCA9685 boots; no servo uncontrolled movement
HC-05 link	Pair/connect/disconnect repeatedly without app crash
Handshake	Robot shows Ready only after valid response
CH0 Base	Small manual movement, release stops, reverse works
CH1 Shoulder	Same; no mechanical binding
CH2 Elbow	Same; verify load/temperature
CH3 Lower Arm	Same
CH4 Wrist	Same
CH5 Gripper	Same
All joints	No brownout, overheating, wiring movement or unexpected motion
Angle calibration	Commanded angle matches physical reference within project tolerance
Pose motion	All target joints move smoothly and reach target
Disconnect test	Communication loss causes safe stop
ESTOP	Immediate software motion stop from control screen

APP QA

- 320 px width minimum: no clipped controls; joysticks remain reachable.
- All primary controls work by touch; no hover dependency.
- Buttons have clear enabled/disabled states and accessible labels.
- Invalid target angle shows a precise reason; never silently clamps without informing the user.
- Bluetooth errors distinguish adapter off, permission denied, not paired, timeout and disconnected.
- Reduced-motion mode disables decorative transitions while preserving workflow.
- No fake telemetry: only display values actually available from Arduino.

RELEASE / PUBLIC WEB GATE

Before calling the product final: connect custom domain → verify HTTPS → add favicon/app icon → remove generator/AI branding → publish Privacy Policy → publish Terms & Conditions → verify legal links inside APK → test clean install → test Bluetooth on target Android phone → perform final robot demonstration.

The locked project requirements explicitly call for custom domain, favicon, privacy policy, terms, no generator branding, no fake metrics/reviews, no AI-slop imagery/copy and no cursor/excessive scroll animation. filecite turn2file2

09 • MASTER BUILD PROMPT — USE AS THE PROJECT BRIEF

You are the senior Flutter/Dart mobile engineer and robotics-control software architect for a small college 6-DOF robotic-arm project. Build a polished Android APK that controls the real robot through Bluetooth Classic HC-05 → Arduino UNO R3 → PCA9685 → six servos. The hardware channel map is CH0 Base/MG995, CH1 Shoulder/MG996R, CH2 Upper Arm/Elbow/MG996R, CH3 Lower Arm/SG90, CH4 Wrist/SG90, CH5 Gripper/SG90.

NON-NEGOTIABLE PRODUCT GOALS

1. The primary function is reliable, game-like manual control of the physical arm.
2. The second primary function is precise six-angle target/pose control: set Base, Shoulder, Elbow, Lower Arm, Wrist and Gripper angles, preview/validate, then explicitly MOVE TO TARGET.
3. Manual control is momentary: press/hold moves; release/center stops. Keep a highly reachable STOP ALL control.
4. Arduino is authoritative for real-time motion, joint limits, emergency stop, connection-loss timeout and coordinated interpolation. Flutter is the UI/Bluetooth/command layer. Firebase is never in the real-time servo path.
5. Without physical feedback sensors, call the displayed value the last commanded/calibrated angle; never imply measured physical accuracy.

VISUAL DESIGN

Use the supplied mobile UI reference only as a visual direction: soft light-blue/cyan/white palette, premium product feel, clean typography, generous spacing, restrained rounded cards, clear hierarchy and polished transitions. Replace the reference robot with our actual 3D robotic-arm model/render. The product must feel like a modern robotics controller. Never use purple gradients, excessive glassmorphism, pill-button-heavy UI, fake reviews, fake metrics, fake counters, vague hero text, emoji as interface icons, AI-slop photos, AI-slop copy, cursor animation, excessive scroll animation, or generator branding. Do not make it look AI-generated or template-generated.

LAUNCH

Create a 3–5 second purposeful intro: actual arm at Home → approaches small block → gripper closes → arm lifts → moves → gripper opens → block is placed → smooth transition to dashboard. Keep it simple, controlled and performant. Respect reduced-motion settings.

APP STRUCTURE

Home: robot visual, connection state, compact live commanded angles, Home, Control shortcut and Stop.
Control: default operating screen with two game-like joysticks; left X=Base, left Y=Shoulder, right X=Elbow, right Y=Lower Arm; dedicated Wrist control; dedicated Gripper control; Fine mode; speed; Stop All.
Pose: six target angles, Current vs Target, limits, preview, Move to Target.
Saved: Home, Pick, Lift, Drop and user-saved poses. Add teach/sequence only after core control is stable.
Settings: Bluetooth/connection, calibration, joint limits, speed defaults, robot info, Privacy Policy and Terms.

BLUETOOTH

Use Bluetooth Classic RFCOMM/SPP, not Web Bluetooth/BLE. Implement permission handling, paired-device listing/discovery where supported, connect/disconnect, connection state, timeout and reconnect. Candidate Flutter package: flutter_classic_bluetooth; verify its current version and compatibility when implementation begins. Use a versioned protocol.
Handshake:
APP → HELLO
UNO → ROBOT_6DOF_READY v1
APP → STATUS?
UNO → STATE ...
APP → READY_ACK
UNO → OK
Only then enable motion controls.

COMMAND PROTOCOL

Use compact line-delimited commands with validation, for example:
SEL:0..5
SPD:1|2|3
POSE:b,s,e,l,w,g
MOVE
STOP
ESTOP
HOME
STATUS?
PING
Manual movement must have an explicit start/stop contract and must never remain active after release or transport loss. Keep commands deterministic and versionable.

MOTION

For target pose, Arduino interpolates all joints from current commanded state to target. Synchronize arrival times instead of independently jumping servos. Enforce calibrated min/max limits on both Flutter and Arduino. Add acceleration/smoothing only after the basic synchronized controller is stable.

CALIBRATION

Each joint has a mechanical home/reference, command offset, minimum, maximum and default speed. Do not assume all servos share the same physical zero. Calibration data is local and may be backed up to Firestore.

SAFETY

No movement before handshake. Stop on release/joystick neutral. Stop on Bluetooth loss. Reject invalid commands. Keep ESTOP visible. Do not invent sensor feedback or telemetry. Test every joint independently before simultaneous operation. Validate the servo power rail and wiring before full-load operation.

FIREBASE

Keep Firebase minimal: saved poses, sequences if implemented, settings and optional history. Do not add unnecessary backend APIs or cloud motion control.

CODE QUALITY

Use a maintainable Flutter structure with core, models, bluetooth, controllers, services, screens and reusable widgets. Keep hardware transport separate from UI. Avoid a giant main.dart. Prefer a simple reactive state-management approach such as Riverpod or Provider. Use reusable widgets for joystick, angle row, status, robot view, emergency stop and pose editor.

EXECUTION ORDER

Freeze hardware map/protocol/safety limits → stabilize Arduino single-joint control → implement pose motion → implement Flutter Bluetooth → build manual control → build angle mode → calibration → saved poses → polish 3D launch → QA → APK → public domain/legal pages → final demonstration.

ACCEPTANCE CRITERIA

The project is complete only when:
- Manual game-like control works reliably on all six axes.
- Release/neutral stops movement.
- Stop All and disconnect safety work.
- Six-angle target mode validates and moves smoothly.
- Displayed angles are honest commanded/calibrated values.
- Bluetooth connection has handshake and useful error states.
- The UI is polished, minimal and clearly robotics-focused.
- No prohibited visual/copy patterns are present.
- The APK installs cleanly on the target Android phone.
- Custom domain, favicon, Privacy Policy and Terms are live before public launch.

MASTER RULE: when a design choice conflicts with the non-negotiable requirements above, preserve control reliability, truthful angle/state reporting, safety, and the restrained visual system before adding features.